



ORIGIN PROTOCOL

OUSD Upgrade Review Security Assessment Report

Version: 2.0

February, 2026

Contents

Introduction	2
Disclaimer	2
Document Structure	2
Overview	2
Security Assessment Summary	3
Scope	3
Approach	3
Coverage Limitations	3
Findings Summary	4
Detailed Findings	5
Summary of Findings	6
CCTP Fee Handling Could Cause Denial Of Service On Withdrawals	7
removeStrategy() May Result In Temporary Accounting Discrepancy	9
Incorrect Yearly APR Calculation Results In Higher Effective Rate	11
Missing Asset Compatibility Check In approveStrategy()	13
Missing Oracle Staleness Check In BridgedWOETHStrategy	15
rebaseThreshold Can Be Bypassed	16
Duplicated Balance Condition Checks In Cross-Chain Strategy	18
Hardcoded Finality Threshold In Message Handler Ignores Configuration	20
Missing CCTP burnToken Validation Could Allow Unexpected Token Types	22
Missing Timestamp Tracking For Remote Balance Updates	24
Missing Zero Amount Validation In requestWithdrawal()	25
Overly Restrictive Finality Threshold Validation Does Not Follow CCTP Specification	26
removeStrategy() Does Not Reset Mint Whitelist Flag	28
_totalValueInVault() Returns The Value Denoted In Asset Decimals	29
withdrawAll() Is No-op When Transfer Pending, Complicating Strategy Removal	30
Miscellaneous General Comments	32
A Test Suite	36
B Vulnerability Severity Classification	37

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Origin Protocol OUSD Upgrade. The review focused solely on the security aspects of the Solidity implementation of the contracts in scope, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Origin Protocol components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

Outputs of automated testing that were developed during this assessment are also included for reference (in the Appendix: [Test Suite](#)).

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Origin Protocol components in scope.

Overview

Origin Protocol's OUSD (Origin Dollar) is a yield-bearing stablecoin that automatically generates returns for holders through rebasing mechanisms. This security review focused on a major upgrade to the OUSD vault system, evaluating two key areas:

- **Vault Contracts Upgrade:** Enhancements to the core vault architecture, including rebase mechanisms, strategy management, and yield distribution systems.
- **Cross-Chain Strategies:** Implementation of cross-chain functionality enabling OUSD to deploy capital across multiple blockchain networks, with a focus on CCTP (Cross-Chain Transfer Protocol) integration for bridging assets between chains.

The review examined critical security aspects including cross-chain message handling, yield calculation accuracy, strategy withdrawal mechanisms, and potential denial-of-service vectors in the upgraded system.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [OriginProtocol/origin-dollar](#) repository.

The scope of this time-boxed review was strictly limited to files modified in the following pull requests:

- [origin-dollar-pr-2714 @ b616bf4](#)
- [origin-dollar-pr-2715 @ 63ff128](#)

The fixes of the identified issues were assessed at their associated fix PRs.

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Solidity.

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

For a more detailed, but non-exhaustive list of examined vectors, see [\[1, 2\]](#).

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: <https://github.com/Cyfrin/aderyn>
- Slither: <https://github.com/trailofbits/slither>
- Mythril: <https://github.com/ConsenSys/mythril>

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 16 issues during this assessment. Categorised by their severity:

- Medium: 3 issues.
- Low: 3 issues.
- Informational: 10 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Origin Protocol components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each issue by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open**: the issue has not been addressed by the project team.
- **Resolved**: the issue was acknowledged by the project team and updates to the affected components(s) have been made to mitigate the related risk.
- **Closed**: the issue was acknowledged by the project team but no further actions have been taken.

Summary of Findings

ID	Description	Severity	Status
OUSD-01	CCTP Fee Handling Could Cause Denial Of Service On Withdrawals	Medium	Closed
OUSD-02	removeStrategy() May Result In Temporary Accounting Discrepancy	Medium	Resolved
OUSD-03	Incorrect Yearly APR Calculation Results In Higher Effective Rate	Medium	Closed
OUSD-04	Missing Asset Compatibility Check In approveStrategy()	Low	Resolved
OUSD-05	Missing Oracle Staleness Check In BridgedW0ETHStrategy	Low	Closed
OUSD-06	rebaseThreshold Can Be Bypassed	Low	Closed
OUSD-07	Duplicated Balance Condition Checks In Cross-Chain Strategy	Informational	Closed
OUSD-08	Hardcoded Finality Threshold In Message Handler Ignores Configuration	Informational	Closed
OUSD-09	Missing CCTP burnToken Validation Could Allow Unexpected Token Types	Informational	Resolved
OUSD-10	Missing Timestamp Tracking For Remote Balance Updates	Informational	Resolved
OUSD-11	Missing Zero Amount Validation In requestWithdrawal()	Informational	Resolved
OUSD-12	Overly Restrictive Finality Threshold Validation Does Not Follow CCTP Specification	Informational	Closed
OUSD-13	removeStrategy() Does Not Reset Mint Whitelist Flag	Informational	Resolved
OUSD-14	_totalValueInVault() Returns The Value Denoted In Asset Decimals	Informational	Resolved
OUSD-15	withdrawAll() Is No-op When Transfer Pending, Complicating Strategy Removal	Informational	Resolved
OUSD-16	Miscellaneous General Comments	Informational	Resolved

OUSD-01	CCTP Fee Handling Could Cause Denial Of Service On Withdrawals		
Asset	contracts/contracts/strategies/crosschain/CrossChainMasterStrategy.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

The `_onTokenReceived()` function incorrectly validates the received USDC balance against the original burnt amount without accounting for CCTP transfer fees, which could cause withdrawal operations to fail if fees are enabled.

In `CrossChainMasterStrategy.sol`, when tokens are received from CCTP, the function checks that the current USDC balance is greater than or equal to the bridged amount.

```
function _onTokenReceived(
    uint256 tokenAmount,
    uint256 feeExecuted,
    bytes memory payload
) internal override {
    uint64 _nonce = lastTransferNonce;

    // ...

    // Send any tokens in the contract to the Vault
    uint256 usdcBalance = IERC20(usdcToken).balanceOf(address(this));
    // Should always have enough tokens
    require(usdcBalance >= tokenAmount, "Insufficient balance"); // @audit Incorrect check

    // ...
}
```

The issue is that `tokenAmount` represents the original amount burnt on the source chain, whilst the actual amount minted on the destination chain is `tokenAmount - feeExecuted`. When CCTP charges a fee:

- Burned amount: 1000 USDC (`tokenAmount`)
- Fee charged: 1 USDC (`feeExecuted`)
- Actual minted: 999 USDC

The check `usdcBalance >= tokenAmount` would require 1000 USDC but only 999 USDC was received, causing the transaction to revert.

Currently, CCTP standard transfers have zero fees, so this issue does not manifest. However, CCTP supports two transfer modes:

1. **Standard transfers** (finality threshold 2000): Currently zero fees, but Circle may enable fees in the future
2. **Fast transfers** (finality threshold 1000): Charge a non-zero fee for faster finality

If Origin decides to enable fast transfers (by setting `minFinalityThreshold` to 1000) or if CCTP enables fees for standard transfers in the future, all withdrawal operations would fail with "Insufficient balance", effectively causing a denial of service until USDC is manually donated to cover the fee shortfall.

The impact of this issue is rated high as it could cause denial of service to protocol operations. The likelihood is rated low as it requires fees to be non-zero.

Recommendations

Modify the balance check to account for fees by comparing the balance against the actual received amount, for example:

```
require(usdcBalance >= tokenAmount - feeExecuted);
```

Additionally, consider documenting the fee handling and ensuring that accounting properly reflects that users will receive slightly less USDC than originally requested if fees are enabled.

Resolution

This finding has been closed as false-positive.

The `tokenAmount` parameter represents the bridged amount with fees already subtracted. Therefore, the check should never revert.

OUSD-02	removeStrategy() May Result In Temporary Accounting Discrepancy		
Asset	contracts/contracts/vault/VaultAdmin.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

`removeStrategy()` introduces a temporary, but material, accounting inconsistency by immediately excluding a cross-chain strategy from vault accounting, before its funds have actually returned.

When a strategy is removed from the vault via `removeStrategy()`, the vault initiates a full withdrawal by calling `withdrawAll()`. For local strategies, this operation is effectively synchronous. However, for cross-chain strategies, `withdrawAll()` triggers an asynchronous cross-chain message, resulting in a non-trivial delay between the withdrawal request and the actual arrival of funds in the vault.

Despite this delay, `removeStrategy()` immediately removes the strategy from `allStrategies`. As a consequence, the strategy's balance is no longer included in the vault's `checkBalance()` calculation immediately after `removeStrategy()` executes, even though the funds are still in transit and economically owned by the vault.

This creates a temporary accounting discrepancy in which the vault's reported total value is understated relative to its true economic value. Once the cross-chain transfer completes and the funds arrive back in the vault, they are again included in `checkBalance()`, and the discrepancy is resolved.

```
function removeStrategy(address _addr) external onlyGovernor {
    // ...

    if (strategyIndex < stratCount) {
        allStrategies[strategyIndex] = allStrategies[stratCount - 1];
        allStrategies.pop(); // @audit strategy is removed, its balance is no longer accounted for in `checkBalance()`

        // Mark the strategy as not supported
        strategies[_addr].isSupported = false;

        // Withdraw all asset
        IStrategy strategy = IStrategy(_addr);
        strategy.withdrawAll(); // @audit delay to receive funds, due to cross-chain call

        emit StrategyRemoved(_addr);
    }
}
```

The impact of this issue is rated medium as this may result in the protocol temporarily being insolvent, potentially blocking withdrawals.

The likelihood is rated medium as it can be triggered by removing a cross-chain strategy that has funds remaining in it.

Recommendations

Remove the call to `withdrawAll()` from `removeStrategy()` and require that all funds must be withdrawn from the strategy before calling `removeStrategy()`.

Resolution

The development team fixed this issue in PR [#2767](#) by adding a non-zero balance check to the `removeStrategy()` method.

OUSD-03	Incorrect Yearly APR Calculation Results In Higher Effective Rate		
Asset	contracts/contracts/vault/VaultAdmin.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Low	Likelihood: High

Description

The `setRebaseRateMax()` function uses simple interest calculations to derive the per-second rebase rate from a yearly APR, but this does not account for the compounding effect that occurs when rebases are executed multiple times per year.

In `VaultAdmin.sol`, the yearly APR is converted to a per-second rate.

```
function setRebaseRateMax(uint256 yearlyApr)
    external
    onlyGovernorOrStrategist
{
    // The old yield will be at the old rate
    _rebase();
    // Change the rate
    uint256 newPerSecond = yearlyApr / 100 / 365 days; // @audit Simple interest formula
    require(newPerSecond <= MAX_REBASE_PER_SECOND, "Rate too high");
    rebasePerSecondMax = newPerSecond.toUint64();
    emit RebasePerSecondMaxChanged(newPerSecond);
}
```

This formula treats the yield as simple interest. However, since `rebase()` is called multiple times (for example, every 8 hours or daily), the yield compounds. The actual effective annual rate will be higher than the intended `yearlyApr`.

For example, if a maximum yearly APR of 20% is intended and `rebase()` is called daily:

- Intended rate: 20%
- Per-day rate using simple calculation: $20\% / 365 = 0.0548\%$ per day
- Actual annual rate with daily compounding: $(1 + 0.000548)^{365} - 1 \approx 22.1\%$

The discrepancy becomes more significant at higher intended rates. For an intended maximum yearly APR of 200%, the actual maximum yearly APR would be approximately 734%.

The likelihood of this issue is rated high as it does not require specific conditions to occur. The impact is rated low since it allows the APR to be higher than intended.

Recommendations

Calculate the correct `newPerSecond` rate offchain using the compound interest formula:

```
newPerSecond = (1 + yearlyApr)^(1/seconds_per_year) - 1
```

Where the exponent accounts for the compounding frequency. Then use the exponential formula when calculating yield in `_nextYield()`.

Alternatively, the development team can leave the implementation as is and adjust the `yearlyApr` parameter lower to match the intended maximum APR, accounting for the compounding effect.

Resolution

The development team has closed the issue with the following comment:

The finding is technically correct. However, given our target APR ranges (typically well below 20%), the difference between the simple interest calculation and accounting for compounding is negligible in practice. We have updated the function to improve clarity in [PR #2785](#): Renamed the parameter from `yearlyApr` to `apr` and updated the NatSpec comment.

OUSD-04	Missing Asset Compatibility Check In approveStrategy()		
Asset	contracts/contracts/vault/VaultAdmin.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

The `approveStrategy()` function does not verify that a strategy supports the vault's asset before approving it, which could lead to funds being allocated to an incompatible strategy and potentially becoming frozen.

In `VaultAdmin.sol`, when approving a new strategy, no asset compatibility check is performed:

```
function approveStrategy(address _addr) external onlyGovernor {
    require(!strategies[_addr].isSupported, "Strategy already approved");
    strategies[_addr] = Strategy({ isSupported: true, _deprecated: 0 }); // @audit No asset support check
    allStrategies.push(_addr);
    emit StrategyApproved(_addr);
}
```

In contrast, `setDefaultStrategy()` properly validates asset compatibility:

```
function setDefaultStrategy(address _strategy)
    external
    onlyGovernorOrStrategist
{
    emit DefaultStrategyUpdated(_strategy);
    // If its a zero address being passed for the strategy we are removing
    // the default strategy
    if (_strategy != address(0)) {
        // Make sure the strategy meets some criteria
        require(strategies[_strategy].isSupported, "Strategy not approved");
        require(
            IStrategy(_strategy).supportsAsset(asset), // @audit Asset support checked here
            "Asset not supported by Strategy"
        );
    }
    defaultStrategy = _strategy;
}
```

Without this check in `approveStrategy()`, a Governor could accidentally approve a strategy that does not support the vault's asset.

The likelihood of this issue is rated low, as it requires operator error to occur. The impact is rated medium as it could potentially lead to frozen funds.

Recommendations

Add an asset support check to `approveStrategy()`, similar to the one used in `setDefaultStrategy()`. This check ensures that only compatible strategies can be approved, reducing the risk of operational errors and protecting against fund freezing scenarios.

Resolution

The development team fixed this issue in PR [#2776](#) by adding a `supportsAsset(asset)` check in `approveStrategy()`.

OUSD-05	Missing Oracle Staleness Check In BridgedWOETHStrategy		
Asset	contracts/contracts/strategies/BridgedWOETHStrategy.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `BridgedWOETHStrategy` contract fetches `wOETH` price data from an oracle without validating the freshness of the data, despite defining a `MAX_PRICE_STALENESS` constant that is never used.

The contract defines a staleness threshold:

```
uint256 public constant MAX_PRICE_STALENESS = 2 days;
```

However, when fetching the oracle price in `_updateWOETHOraclePrice()`, there is no check to ensure the data is recent:

```
function _updateWOETHOraclePrice() internal returns (uint256) {
    // WETH price per unit of bridged wOETH
    uint256 oraclePrice = oracle.price(address(bridgedWOETH)); // @audit No staleness check

    // 1 wOETH > 1 WETH, always
    require(oraclePrice > 1 ether, "Invalid wOETH value");

    uint128 oraclePrice128 = oraclePrice.toUint128();

    // ... rest of the function
}
```

The likelihood of this issue is rated low as the oracle should not return stale data in normal conditions. The impact is rated low since a stale price may be used.

Recommendations

Implement a staleness check when fetching oracle data. The oracle should return both a price and timestamp, allowing verification that the data is recent.

If the oracle interface does not provide timestamp information, consider either removing the unused `MAX_PRICE_STALENESS` constant or working with the oracle provider to add staleness checking capabilities.

Resolution

The development team has closed the issue with the following comment:

Staleness does not affect this strategy: only the Guardian can deposit or withdraw, and we have checks ensuring the oracle price only increases and stays within bounds. We removed the unused `MAX_PRICE_STALENESS` constant for cleanliness in [PR #2784](#).

OUSD-06	rebaseThreshold Can Be Bypassed		
Asset	contracts/contracts/vault/VaultCore.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

In the `_mint()` function in `VaultCore`, `_rebase()` is called if the minted amount is larger than `rebaseThreshold`. This ensures that any pending rewards are distributed through the drip mechanism before a mint occurs. However, this threshold can be bypassed by minting in multiple smaller batches, allowing an attacker to mint large amounts of `oTokens` without triggering a rebase.

```
function _mint(uint256 _amount) internal virtual {
    // ...

    // Scale amount to 18 decimals
    uint256 scaledAmount = _amount.scaleBy(18, assetDecimals);

    emit Mint(msg.sender, scaledAmount);

    // Rebase must happen before any transfers occur.
    if (!rebasePaused && scaledAmount >= rebaseThreshold) { // @audit can be bypassed by minting in small amounts
        _rebase();
    }

    // ... mint oTokens
}
```

For example, consider a scenario where `rebase()` has not been called in a long time and the protocol has collected a significant amount of rewards. In this case, an attacker can mint a large amount of `oTokens` using this vulnerability, call `rebase()` afterwards, and then freeloader off these rewards for the duration of the drip. In contrast, if the `rebaseThreshold` was triggered whilst minting as intended, then the potential profit would be much smaller.

The impact of this issue is rated low as the potential profits are small. The likelihood is rated low as this attack is capital intensive and may not be profitable compared to the market yield.

Recommendations

Consider modifying the `rebaseThreshold` mechanism, for example by keeping track of the amount of tokens minted since the last rebase.

Resolution

The development team has closed the issue with the following comment:

The amount of capital at risk is at maximum the amount of total rewards accrued since the last rebase (the underlying principle funds are not affected). Since multiple mints are required there is only so much `oTokens` an attacker can mint per a single transaction before reaching the max gas per block limit. Once the attacker has

minted oTokens there is not a fast way to efficiently off-load them. Choices are instant redeem with a higher fee, async redemption or offloading on a DEX. Which makes a yield stealing attack risky and unpredictable. This and the previous point prevent an attack to be done using a flash loan. To efficiently solve the threshold issue, the fix would probably cause increased complexity and higher gas usage on mints.

For those reasons we currently do not see a need to address this issue.

OUSD-07	Duplicated Balance Condition Checks In Cross-Chain Strategy	
Asset	contracts/contracts/strategies/crosschain/CrossChainMasterStrategy.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

The `CrossChainMasterStrategy` contract contains duplicated balance condition checks that could be refactored to improve code maintainability and reduce the risk of inconsistencies.

There are two instances of duplication:

Duplication 1: Minimum deposit check

In `depositAll()` at line [96]:

```
function depositAll() external override onlyVault nonReentrant {
    uint256 balance = IERC20(usdcToken).balanceOf(address(this));
    // Deposit if balance is greater than 1 USDC
    if (balance >= 1e6) { // @audit First check
        _deposit(usdcToken, balance);
    }
}
```

Then duplicated in `_deposit()` at line [239]:

```
function _deposit(address _asset, uint256 depositAmount) internal virtual {
    require(_asset == usdcToken, "Unsupported asset");
    require(pendingAmount == 0, "Unexpected pending amount");
    // Deposit at least 1 USDC
    require(depositAmount >= 1e6, "Deposit amount too small"); // @audit Duplicate check
    require(
        depositAmount <= MAX_TRANSFER_AMOUNT,
        "Deposit amount too high"
    );
    // ... rest of function
}
```

Duplication 2: Minimum withdrawal check with different behaviour

In `withdrawAll()` at line [120]:

```
function withdrawAll() external override onlyVaultOrGovernor nonReentrant {
    if (isTransferPending()) {
        // Do nothing if there is a pending transfer
        return;
    }

    // Withdraw everything in Remote strategy
    uint256 _remoteBalance = remoteStrategyBalance;
    if (_remoteBalance < 1e6) { // @audit Silent return
        // Do nothing if there is less than 1 USDC in the Remote strategy
        return;
    }

    _withdraw(
        usdcToken,
        _remoteBalance > MAX_TRANSFER_AMOUNT
        ? MAX_TRANSFER_AMOUNT
        : _remoteBalance
    );
}
```

Then in `_withdraw()` at line [273]:

```
function _withdraw(address _asset, uint256 _amount) internal virtual {
    require(_asset == usdcToken, "Unsupported asset");
    // Withdraw at least 1 USDC
    require(_amount >= 1e6, "Withdraw amount too small"); // @audit Reverts instead
    require(
        _amount <= remoteStrategyBalance,
        "Withdraw amount exceeds remote strategy balance"
    );
    // ... rest of function
}
```

The first check in `depositAll()` and `withdrawAll()` is actually unnecessary since the internal functions already validate the amount. However, these checks change the behaviour from reverting to silently returning, which may be intentional for the vault-facing API.

Recommendations

Remove the duplicate checks from `depositAll()` and `withdrawAll()` if silent returns are not required. Alternatively, keep the duplicate checks but extract the constant to a single location.

Resolution

The development team has closed the issue with the following comment:

depositAll() and withdrawAll() check for the same amount as _deposit() and _withdraw(), but the consequence of the condition is different. The former functions need to succeed without reverting because of the requirements of our operations, whilst the latter should revert when a condition is not met. We moved the 1e6 constant to a new constant in [PR #2780](#).

OUSD-08	Hardcoded Finality Threshold In Message Handler Ignores Configuration	
Asset	contracts/contracts/strategies/crosschain/AbstractCCTPIntegrator.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

The `handleReceiveFinalizedMessage()` function uses a hardcoded finality threshold of 2000 instead of using the configurable `minFinalityThreshold` variable, creating an inconsistency in the contract's finality handling.

In `AbstractCCTPIntegrator.sol`, there is a configurable `minFinalityThreshold` variable:

```
/**
 * @notice Minimum finality threshold
 *         Can be 1000 (safe, after 1 epoch) or 2000 (finalized, after 2 epochs).
 *         Ref: https://developers.circle.com/cctp/technical-guide#finality-thresholds
 * @dev    When configuring the contract for fast transfer we should check the available
 *         allowance of USDC that can be bridged using fast mode:
 *         wget https://iris-api.circle.com/v2/fastBurn/USDC/allowance
 */
uint16 public minFinalityThreshold;
```

However, `handleReceiveFinalizedMessage()` uses a hardcoded value:

```
function handleReceiveFinalizedMessage(
    uint32 sourceDomain,
    bytes32 sender,
    uint32 finalityThresholdExecuted,
    bytes memory messageBody
) external override onlyCCTPMessageTransmitter returns (bool) {
    // Make sure the finality threshold at execution is at least 2000
    require(
        finalityThresholdExecuted >= 2000, // @audit Hardcoded 2000
        "Finality threshold too low"
    );

    return _handleReceivedMessage(sourceDomain, sender, messageBody);
}
```

Meanwhile, `handleReceiveUnfinalizedMessage()` correctly uses the configured value:

```
function handleReceiveUnfinalizedMessage(  
    uint32 sourceDomain,  
    bytes32 sender,  
    uint32 finalityThresholdExecuted,  
    bytes memory messageBody  
) external override onlyCCTPMessageTransmitter returns (bool) {  
    // Make sure the contract is configured to handle unfinalized messages  
    require(  
        minFinalityThreshold == 1000, // @audit Uses configured value  
        "Unfinalized messages are not supported"  
    );  
    // Make sure the finality threshold at execution is at least 1000  
    require(  
        finalityThresholdExecuted >= 1000,  
        "Finality threshold too low"  
    );  
  
    return _handleReceivedMessage(sourceDomain, sender, messageBody);  
}
```

This creates an inconsistency where:

1. `handleReceiveFinalizedMessage()` always requires 2000 regardless of configuration
2. `handleReceiveUnfinalizedMessage()` respects the configured `minFinalityThreshold`
3. The configuration variable cannot affect finalised message handling

Recommendations

Use the configured threshold for consistency. Alternatively, consider documenting the intentional hardcoding.

Resolution

The development team has closed the issue with the following comment:

CCTP's Receiver contracts have two methods: `handleReceiveFinalizedMessage` for finalized messages and `handleReceiveUnfinalizedMessage` for unfinalized messages. A message is considered finalized in CCTP if two epochs have passed (and finality threshold is 2000 or greater than 2000). If it is 1000 or anything below 2000, it is considered unfinalized but safe. Ideally we can blindly trust CCTP Message Transmitter, but the check `finalityThreshold >= 2000` exists in `handleReceiveFinalizedMessage` as a means of defensive programming.

OUSD-09	Missing CCTP burnToken Validation Could Allow Unexpected Token Types	
Asset	contracts/contracts/strategies/crosschain/AbstractCCTPIntegrator.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The CCTP message relay function does not validate which token was burnt and minted, potentially allowing unexpected token types to be accepted if CCTP expands its supported token set beyond USDC in the future.

CCTP currently supports bridging USDC, but also supports USYC and may add more tokens in the future. The burn message body contains a `burnToken` field that specifies which token was burnt on the source chain. However, the `relay()` function in `AbstractCCTPIntegrator.sol` does not validate this field:

```
function relay(bytes memory message, bytes memory attestation)
    external
    onlyOperator
{
    (
        uint32 version,
        uint32 sourceDomainID,
        address sender,
        address recipient,
        bytes memory messageBody
    ) = message.decodeMessageHeader();

    // ... version and domain checks

    version = messageBody.extractUint32(BURN_MESSAGE_V2_VERSION_INDEX);

    bool isBurnMessageV1 = sender == address(cctpTokenMessenger);

    if (isBurnMessageV1) {
        // Handle burn message
        require(
            version == 1 &&
            messageBody.length >= BURN_MESSAGE_V2_HOOK_DATA_INDEX,
            "Invalid burn message"
        );

        // Address of caller of depositForBurn (or depositForBurnWithCaller) on source domain
        sender = messageBody.extractAddress(
            BURN_MESSAGE_V2_MESSAGE_SENDER_INDEX
        );

        recipient = messageBody.extractAddress(
            BURN_MESSAGE_V2_RECIPIENT_INDEX
        );

        // @audit Missing: validation of burnToken field
    } else {
        // ... custom message handling
    }

    // ... rest of relay logic
}
```

According to the CCTP burn message specification, the message body contains the `burnToken` address which indicates which token was burnt. Without validating this field, the contract could potentially accept and process messages for

tokens other than USDC.

Currently, this is not exploitable because:

- The contract only accepts messages from the designated `peerStrategy` address
- The peer strategy can only send USDC
- CCTP on the current deployment only bridges USDC

However, for defence in depth and future compatibility, the `burnToken` should be explicitly validated to ensure only expected tokens are processed.

This issue is rated as informational as it represents a missing defence-in-depth validation rather than an active vulnerability.

Recommendations

Add validation to check the `burnToken` field in the burn message body.

Note, ensure the validation accounts for the fact that `burnToken` represents the token address on the source chain, which may differ from the local chain's USDC address.

Resolution

The development team fixed this issue in [#2782](#) by implementing a check on the `burnToken` field.

OUSD-10	Missing Timestamp Tracking For Remote Balance Updates	
Asset	contracts/contracts/strategies/crosschain/CrossChainMasterStrategy.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The `CrossChainMasterStrategy` does not track when the `remoteStrategyBalance` was last updated, which prevents implementing safety checks based on balance freshness and makes it difficult to monitor cross-chain synchronisation health.

In `CrossChainMasterStrategy.sol`, the remote balance is updated without timestamp tracking. This means it is not possible to determine if the remote balance is stale or not.

```
function _processBalanceCheckMessage(bytes memory message)
    internal
    virtual
{
    // Decode the message
    (uint64 nonce, uint256 balance, bool transferConfirmation) = message
        .decodeBalanceCheckMessage();
    // Get the last cached nonce
    uint64 _lastCachedNonce = lastTransferNonce;

    if (nonce != _lastCachedNonce) {
        // If nonce is not the last cached nonce, it is an outdated message
        // Ignore it
        return;
    }

    // ... transfer processing logic

    // At this point update the strategy balance the balanceCheck message is either:
    // - a confirmation of a deposit / withdrawal
    // - a message that updates balances when no deposit / withdrawal is in progress
    remoteStrategyBalance = balance; // @audit No timestamp recorded
    emit RemoteStrategyBalanceUpdated(balance);
}
```

Recommendations

Consider adding timestamp tracking for remote balance updates.

Resolution

The development team fixed this issue in PR [#2786](#) by adding a staleness check to balance check messages.

OUSD-11	Missing Zero Amount Validation In <code>requestWithdrawal()</code>	
Asset	contracts/contracts/vault/VaultCore.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The `requestWithdrawal()` function does not validate that the withdrawal amount is greater than zero, allowing users to create withdrawal requests with zero amounts.

In `VaultCore.sol`, the function accepts any `_amount` value without checking if it is zero:

```
function requestWithdrawal(uint256 _amount)
    external
    virtual
    whenNotCapitalPaused
    nonReentrant
    returns (uint256 requestId, uint256 queued)
{
    require(withdrawalClaimDelay > 0, "Async withdrawals not enabled");

    // The check that the requester has enough OToken is done in to later burn call
    // @audit No check for _amount > 0

    requestId = withdrawalQueueMetadata.nextWithdrawalIndex;
    queued =
        withdrawalQueueMetadata.queued +
        _amount.scaleBy(assetDecimals, 18);

    // Store the next withdrawal request
    withdrawalQueueMetadata.nextWithdrawalIndex = SafeCast.toUint128(
        requestId + 1
    );
    // ... continues storing state
```

Recommendations

Consider adding a validation check at the beginning of the function to ensure the withdrawal amount is non-zero. This check follows the same pattern already used in the `mint()` function in `VaultCore`.

Resolution

The development team fixed this issue in PR [#2777](#) by adding validation to reject zero-amount withdrawal requests in `requestWithdrawal()`.

OUSD-12	Overly Restrictive Finality Threshold Validation Does Not Follow CCTP Specification	
Asset	contracts/contracts/strategies/crosschain/AbstractCCTPIntegrator.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

The `_setMinFinalityThreshold()` function enforces that the finality threshold must be exactly 1000 or 2000, but CCTP documentation indicates that any value below 1000 is treated as 1000 and any value above 1000 is treated as 2000.

In `AbstractCCTPIntegrator.sol`, the validation is overly strict:

```
function _setMinFinalityThreshold(uint16 _minFinalityThreshold) internal {
    // 1000 for fast transfer and 2000 for standard transfer
    require(
        _minFinalityThreshold == 1000 || _minFinalityThreshold == 2000, // @audit Too strict
        "Invalid threshold"
    );

    minFinalityThreshold = _minFinalityThreshold;
    emit CCTPMinFinalityThresholdSet(_minFinalityThreshold);
}
```

According to the CCTP documentation referenced in the code comments:

"Only two finality thresholds are supported. Any minFinalityThreshold value below 1000 is treated as 1000, and any value above 1000 is treated as 2000."

This means CCTP will accept and map any value to one of these two thresholds, but the contract rejects values that do not exactly match 1000 or 2000. Whilst the current implementation works correctly for the standard use cases, it unnecessarily restricts configuration flexibility.

For example, if governance wanted to set the threshold to 500 (which CCTP would treat as 1000) or 1500 (which CCTP would treat as 2000), the transaction would revert even though CCTP would handle these values correctly.

Recommendations

Update the validation to follow CCTP's actual specification.

Alternatively, if strict control is desired, add informative documentation explaining why only exact values are accepted whilst CCTP would technically accept others.

Resolution

The development team has closed the issue with the following comment:

Although CCTP supports a broader range of message finality values, the current contract behavior is intentional. We have deliberately chosen to support only two finality threshold values: 1000 and 2000.

OUSD-13	removeStrategy() Does Not Reset Mint Whitelist Flag	
Asset	contracts/contracts/vault/VaultAdmin.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The `removeStrategy()` function does not reset the `isMintWhitelistedStrategy` mapping when removing a strategy, leaving stale whitelist data in contract state. Due to sufficient checks in `mintForStrategy()`, a removed strategy cannot mint any tokens. However, when a removed strategy is re-added, it would still be whitelisted and instantly be able to mint tokens.

In `VaultAdmin.sol`, when a strategy is removed, the whitelist flag is not cleared:

```
function removeStrategy(address _addr) external onlyGovernor {
    // ...

    if (strategyIndex < stratCount) {
        allStrategies[strategyIndex] = allStrategies[stratCount - 1];
        allStrategies.pop();

        // Mark the strategy as not supported
        strategies[_addr].isSupported = false;

        // @audit Missing: isMintWhitelistedStrategy[_addr] = false;

        // Withdraw all asset
        IStrategy strategy = IStrategy(_addr);
        strategy.withdrawAll();

        emit StrategyRemoved(_addr);
    }
}
```

Recommendations

Clear the whitelist flag when removing a strategy to maintain consistent state.

Resolution

The development team fixed this issue in PR [#2778](#). The `removeStrategy()` function now resets the `isMintWhitelisted` flag when removing a strategy.

OUSD-14	_totalValueInVault() Returns The Value Denoted In Asset Decimals	
Asset	contracts/contracts/vault/VaultCore.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The `_totalValueInVault()` function returns the value denoted in the asset's decimals, not in 18 decimals as the comment indicates. In the case of `oUSD`, the asset (USDC) only has 6 decimals, so the difference may be significant.

```
/**
 * // ...
 * @return value Total value in USD/ETH (1e18) // @audit incorrect, value is denoted in asset's decimals
 */
function _totalValueInVault()
    internal
    view
    virtual
    returns (uint256 value)
{
    value = IERC20(asset).balanceOf(address(this));
}
```

Seeing as this function is unused, there is no real impact and this issue is rated informational.

Recommendations

Consider refactoring `_totalValueInVault()`, or removing it if it is not needed.

Resolution

The development team fixed this issue in PR [#2779](#) by removing the unused `_totalValueInVault()` function from `VaultCore`.

OUSD-15	withdrawAll() Is No-op When Transfer Pending, Complicating Strategy Removal	
Asset	contracts/contracts/strategies/crosschain/CrossChainMasterStrategy.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The `withdrawAll()` function in `CrossChainMasterStrategy` silently returns without withdrawing funds when a cross-chain transfer is pending, which can complicate strategy removal operations and require manual intervention to recover funds.

In `CrossChainMasterStrategy.sol`, the `withdrawAll()` implementation checks for pending transfers:

```
function withdrawAll() external override onlyVaultOrGovernor nonReentrant {
    if (isTransferPending()) {
        // Do nothing if there is a pending transfer
        return; // @audit Silent no-op
    }

    // ...
}
```

This design choice creates operational complexity when used with the vault's `removeStrategy()` function. The `removeStrategy()` function in `VaultAdmin.sol` calls `withdrawAll()` and then immediately removes the strategy, meaning that funds may remain in the strategy. If this occurs, manual intervention would be required to recover the funds.

```
function removeStrategy(address _addr) external onlyGovernor {
    require(strategies[_addr].isSupported, "Strategy not approved");
    require(defaultStrategy != _addr, "Strategy is default for asset");

    // ... find strategy index

    if (strategyIndex < stratCount) {
        allStrategies[strategyIndex] = allStrategies[stratCount - 1];
        allStrategies.pop();

        // Mark the strategy as not supported
        strategies[_addr].isSupported = false;

        // Withdraw all asset
        IStrategy strategy = IStrategy(_addr);
        strategy.withdrawAll(); // @audit May be a no-op

        emit StrategyRemoved(_addr);
    }
}
```

Recommendations

Consider emitting an event when a withdrawal is skipped due to a pending transfer, so that this behaviour is noticed and the funds can be recovered.

Resolution

The development team fixed this issue in PR [#2781](#) by adding an event that is emitted when `withdrawAll()` is skipped due to a pending transfer.

OUSD-16	Miscellaneous General Comments
Asset	All contracts
Status	Resolved: See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. IWETH9 Interface Used For Non-WETH Assets

Related Asset(s): *contracts/contracts/vault/VaultStorage.sol*

The `VaultStorage` constructor uses the `IWETH9` interface to call `decimals()` on the asset token, even though the asset may be USDC or other non-WETH tokens, creating unnecessary coupling to the WETH interface and potential confusion.

```
import { IWETH9 } from "../interfaces/IWETH9.sol";

// ... contract definition ...

constructor(address _asset) {
    uint8 _decimals = IWETH9(_asset).decimals(); // @audit Using IWETH9 for non-WETH assets
    require(_decimals <= 18, "invalid asset decimals");
    asset = _asset;
    assetDecimals = _decimals;
}
```

Consider using the standard ERC20 interface for retrieving decimals.

2. Inconsistent Use Of SafeERC20 In BridgedWOETHStrategy

Related Asset(s): *contracts/contracts/strategies/BridgedWOETHStrategy.sol*

The `BridgedWOETHStrategy` contract imports OpenZeppelin's `SafeERC20` library but uses unsafe `transfer` and `transferFrom` methods directly on ERC20 tokens instead of using the safe versions, leading to inconsistent and potentially unsafe token handling patterns.

The contract imports and declares usage of `SafeERC20`:

```
import { IERC20, SafeERC20, InitializableAbstractStrategy } from "../utils/InitializableAbstractStrategy.sol";

contract BridgedWOETHStrategy is InitializableAbstractStrategy {
    using StableMath for uint256;
    using StableMath for uint128;
    using SafeCast for uint256;
    using SafeERC20 for IERC20; // @audit Declared but not consistently used
```

However, throughout the contract, token transfers use the unsafe methods with Slither suppressions:

```
// In depositBridgedWOETH() at line 172-173
// slither-disable-next-line unchecked-transfer unused-return
oethb.transfer(msg.sender, oethToMint); // @audit Should use safeTransfer

// At line 176-177
// slither-disable-next-line unchecked-transfer unused-return
bridgedWOETH.transferFrom(msg.sender, address(this), woethAmount); // @audit Should use safeTransferFrom

// In withdrawBridgedWOETH() at line 201-203
// slither-disable-next-line unchecked-transfer unused-return
bridgedWOETH.transfer(msg.sender, woethAmount); // @audit Should use safeTransfer

// At line 205-207
// slither-disable-next-line unchecked-transfer unused-return
oethb.transferFrom(msg.sender, address(this), oethToBurn); // @audit Should use safeTransferFrom
```

Whilst the specific tokens used (oethb, bridgedWOETH) may be standard-compliant implementations controlled by the development team, using SafeERC20 consistently is a best practice that provides defence in depth.

Use the SafeERC20 wrapper consistently throughout the contract. Alternatively, if unsafe transfers are intentionally used for gas optimisation with known-safe tokens, remove the SafeERC20 import and document this design decision.

3. Missing Documentation For _nextYield() Function

Related Asset(s): contracts/contracts/vault/VaultCore.sol

The internal function _nextYield() lacks in-line comments or NatSpec documentation explaining its complex yield calculation logic, making the code difficult to understand and maintain.

Add comprehensive NatSpec documentation to the function.

4. Unresolved TODO Comment In burnForStrategy() Function

Related Asset(s): contracts/contracts/vault/VaultCore.sol

The burnForStrategy() function contains an unresolved TODO comment questioning whether specific documentation should be removed, indicating incomplete code review or cleanup.

```
/**
 * @notice Burn OTokens for an allowed Strategy
 * @param _amount Amount of OToken to burn
 *
 * TODO: Maybe this is a comment that we can remove now? // @audit Unresolved TODO
 * @dev Notice: can't use `nonReentrant` modifier since the `redeem` function could
 * require withdrawal on an AMO strategy and that one can call `burnForStrategy`
 * while the execution of the `redeem` has not yet completed -> causing a `nonReentrant` collision.
 *
 * Also important to understand is that this is a limitation imposed by the test suite.
 * Production / mainnet contracts should never be configured in a way where mint/redeem functions
 * that are moving funds between the Vault and end user wallets can influence strategies
 * utilizing this function.
 */
function burnForStrategy(uint256 _amount)
    external
    virtual
    whenNotCapitalPaused
{
    // ...
}
```

Review the TODO comment and remove it if it is unnecessary.

5. Confusing Variable Name oUSD Used For Multiple Token Types

Related Asset(s): contracts/contracts/vault/VaultStorage.sol

The state variable oUSD is used to represent any OToken (OUSD, OETH, or others), which can cause confusion during development and maintenance since the name suggests it specifically represents OUSD tokens.

In VaultStorage.sol, the variable is defined as:

```
/// @dev Address of the OToken token. eg OUSD or OETH.
OUSD public oUSD; // @audit Confusing name when it may be OETH or other tokens
```

Whilst the comment clarifies that this could be "OUSD or OETH", the variable name itself strongly implies it is specifically the OUSD token.

Rename the variable to something more generic that accurately reflects its purpose, for example to `oToken`

6. IERC4626 Interface Imported From Local File Instead Of OpenZeppelin Library

Related Asset(s): `contracts/contracts/strategies/crosschain/CrossChainRemoteStrategy.sol`

The `CrossChainRemoteStrategy` contract imports the `IERC4626` interface from a local file instead of using the OpenZeppelin library directly. This is because the project currently uses an older OpenZeppelin version (4.4.2) which does not include this interface.

```
import { SafeERC20 } from "openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import { IERC20 } from "openzeppelin/contracts/token/ERC20/IERC20.sol";
import { IERC4626 } from "openzeppelin/contracts/interfaces/IERC4626.sol"; // @audit Local import
```

Whilst this approach works, maintaining local copies of standard interfaces can lead to inconsistencies and increased maintenance burden.

Consider updating the OpenZeppelin dependency to a more recent version that includes the `IERC4626` interface, allowing the contract to import it directly from the OpenZeppelin library.

7. Unused `_recipient` Parameter In `CrossChainMasterStrategy.withdraw()`

Related Asset(s): `contracts/contracts/strategies/crosschain/CrossChainMasterStrategy.sol`

The `withdraw()` function in `CrossChainMasterStrategy` accepts a `_recipient` parameter but does not use it. Tokens are always sent to `vaultAddress` as hardcoded in the `_onTokenReceived()` function on line [224].

```
function withdraw(
    address _recipient,
    address _asset,
    uint256 _amount
) external override onlyVault nonReentrant {
    require(_recipient == vaultAddress, "Only Vault can withdraw"); // @audit _recipient not actually used
    // ...
}
```

The actual token transfer happens in `_onTokenReceived()`:

```
function _onTokenReceived(
    uint256 tokenAmount,
    uint256 feeExecuted,
    bytes memory payload
) internal override {
    // ...
    uint256 usdcBalance = IERC20(usdcToken).balanceOf(address(this));
    IERC20(usdcToken).safeTransfer(vaultAddress, usdcBalance); // @audit Hardcoded destination
    // ...
}
```

Since the `onlyVault` modifier already restricts who can call this function, the `_recipient` parameter serves no functional purpose unless it is for backwards compatibility.

Consider either removing the `_recipient` parameter if it is not needed for interface compatibility, or documenting why it must always equal `vaultAddress`.

8. Unused `_recipient` Parameter In `CrossChainRemoteStrategy.withdraw()`

Related Asset(s): `contracts/contracts/strategies/crosschain/CrossChainRemoteStrategy.sol`

Similar to the issue in `CrossChainMasterStrategy`, the `withdraw()` function in `CrossChainRemoteStrategy` accepts a `_recipient` parameter but requires it to always be `address(this)`.

```
function withdraw(
    address _recipient,
    address _asset,
    uint256 _amount
) external override onlyVault nonReentrant {
    require(_recipient == address(this), "Invalid recipient"); // @audit Must always be address(this)
    // ...
}
```

The parameter is validated but not meaningfully used since it must always equal the contract's own address.

Consider either removing the `_recipient` parameter if it is not needed for interface compatibility, or documenting why it must always equal `address(this)`.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

Several of these issues were fixed in PRs [#2788](#) and [#2787](#), the remaining issues were closed with the following comments:

2. *Inconsistent use is intentional as, we can be sure that oethb and bridgedWOETH implement ERC20 standard correctly. The safeTransfer is used where a non standard asset needs to be rescued from the contract. We can't know if that one implements ERC20 correctly.*

6. *Thank you for the suggestion. Updating the OpenZeppelin library version is a significant undertaking that could introduce breaking changes and new security considerations. For these reasons, we've decided not to proceed with an upgrade at this time.*

7. *Thank you for the suggestion. This is a defensive programming measure we'd prefer to keep.*

Appendix A Test Suite

A non-exhaustive list of tests were constructed to aid this security review and are given along with this document. The forge framework was used to perform these tests and the output is given below.

```
Ran 2 tests for test/tests-local/PR2715/CrossChainRemoteStrategy.t.sol:CrossChainRemoteStrategyTestSigP
[PASS] test_sigp_deposit_to_platform(uint256) (runs: 1000, μ: 182657, ~: 182729)
[PASS] test_sigp_deposit_withdraw_platform(uint256) (runs: 1000, μ: 252306, ~: 252376)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 694.91ms (681.35ms CPU time)

Ran 6 tests for test/tests-local/PR2714/VaultCore.t.sol:VaultCoreTestSigP
[PASS] test_sigp_burn_for_strategy(uint256,uint256) (runs: 1000, μ: 193565, ~: 192057)
[PASS] test_sigp_burn_for_strategy_unsupported(uint256,uint256) (runs: 1000, μ: 428768, ~: 428956)
[PASS] test_sigp_mint_for_strategy(uint256) (runs: 1000, μ: 175562, ~: 175693)
[PASS] test_sigp_mint_for_strategy_unsupported(uint256) (runs: 1000, μ: 309664, ~: 309780)
[PASS] test_sigp_mint_only(uint256) (runs: 1000, μ: 243802, ~: 245464)
[PASS] test_sigp_request_claim_withdrawal(uint256,uint256) (runs: 1000, μ: 532966, ~: 533908)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 1.24s (2.01s CPU time)

Ran 5 tests for test/tests-local/PR2715/CrossChainMasterStrategy.t.sol:CrossChainMasterStrategyTestSigP
[PASS] test_Revert_sigp_deposit_double_no_forward(uint256) (runs: 1000, μ: 670929, ~: 671018)
[PASS] test_Revert_sigp_withdraw_with_fees(uint256) (runs: 1000, μ: 1619787, ~: 1619787)
[PASS] test_sigp_deposit(uint256) (runs: 1000, μ: 1573457, ~: 1576512)
[PASS] test_sigp_deposit_then_send_balance_update(uint256) (runs: 1000, μ: 1832394, ~: 1832481)
[PASS] test_sigp_withdraw_no_fees(uint256) (runs: 1000, μ: 2773862, ~: 2773862)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 6.92s (18.82s CPU time)

Ran 3 test suites in 6.92s (8.85s CPU time): 13 tests passed, 0 failed, 0 skipped (13 total tests)
```

Appendix B Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

References

- [1] Sigma Prime. Solidity Security. Blog, 2018, Available: <https://blog.sigmaprime.io/solidity-security.html>. [Accessed 2018].
- [2] NCC Group. DASP - Top 10. Website, 2018, Available: <http://www.dasp.co/>. [Accessed 2018].

σ'